CSP-S 2020 (Senior)

2020-11-07, 14:30–18:30 (4 hours)

Second round of the 2020 CCF Certified Software Professional (non-professional level), Senior group. There are four problems, listed in contest order.

Problem 1: Julian Day (儒略日)

Problem file name: `julian`. Time limit: 1 s. Memory limit: 256 MB.

Description

To simplify calculations, astronomers use the Julian day to express time. The Julian day is defined as the number of days elapsed from noon (12 o'clock) on January 1, 4713 BC to a given moment, with fractions of a day expressed as decimals. Using this astronomical calendar, every moment is mapped uniformly onto the number line, so that the differences between moments can be computed easily.

Now, given a Julian day without a fractional part, please compute the date of the Gregorian calendar that corresponds to this Julian day (which is necessarily noon, 12 o'clock, of some day).

The calendar we use today is the Gregorian calendar, which was obtained in 1582 AD when Pope Gregory XIII modified the earlier Julian calendar (note: the Julian calendar has no direct relation to the Julian day). Concretely, dates of the current calendar are computed according to the following rules:

1. On and after October 15, 1582 AD: the Gregorian calendar applies. In every year, January has 31 days, February has 28 or 29 days, March has 31 days, April has 30 days, May has 31 days, June has 30 days, July has 31 days, August has 31 days, September has 30 days, October has 31 days, November has 30 days, and December has 31 days. February has 29 days in a leap year and 28 days in a common year. A year is a leap year when the year number is a multiple of 400, or when the year number is a multiple of 4 but not a multiple of 100.

2. From October 5, 1582 AD (inclusive) to October 14, 1582 AD (inclusive): these dates do not exist; they were deleted, and the day after October 4 of that year is October 15.

3. On and before October 4, 1582 AD: the Julian calendar applies. The number of days in each month is the same as in the Gregorian calendar, but a year is a leap year whenever the year number is a multiple of 4.

4. Although the Julian calendar was only introduced in 45 BC, and went through several adjustments in its early period, today people are accustomed to extrapolating all times before

October 4, 1582 according to the final rules of the Julian calendar. Note that the year 0 AD does not exist, i.e., the year after 1 BC is 1 AD. Therefore the years 1 BC, 5 BC, 9 BC, 13 BC, ..., and so on, are to be regarded as leap years.

Input

The input file is named `julian.in`.

The first line contains an integer Q, the number of queries.

Each of the next Q lines contains a non-negative integer r_i, representing a Julian day.

Output

The output file is named `julian.out`.

For each Julian day r_i, output one line containing a string s_i that represents the date. There are Q lines in total.

The format of s_i is as follows:

1. If the year is AD, the output format is `Day Month Year`. Here the day (Day), month (Month) and year (Year) are all written without leading zeros, separated by single spaces. For example, noon (12 o'clock) of November 7, 2020 AD is output as `7 11 2020`.
2. If the year is BC, the output format is `Day Month Year BC`. Here the year (Year) is the numerical value of that year, and the rest is the same as for AD. For example, noon (12 o'clock) of February 1, 841 BC is output as `1 2 841 BC`.

Sample 1 input

```
3
10
100
1000
```

Sample 1 output

```
11 1 4713 BC
10 4 4713 BC
27 9 4711 BC
```

Sample 2 input

```
3
2000000
3000000
4000000
```

Sample 2 output

```
14 9 763
15 8 3501
12 7 6239
```

Sample 3

See `julian/julian3.in` and `julian/julian3.ans` in the contestant's directory. (On QOJ these are given as downloadable files `ex_julian3.in` and `ex_julian3.ans`; the QOJ attachment also contains an additional pair `ex_julian4.in` and `ex_julian4.ans`.)

Constraints

Test case number	$Q =$	$r_i \leq$
1	1000	365
2		10^4
3		10^5
4	10000	3×10^5
5		2.5×10^6
6	10^5	
7		5×10^6
8		10^7
9		10^9
10		the year of the answer does not exceed 10^9

(In the original table, the cell $Q = 1000$ spans test cases 1–3, $Q = 10000$ spans test cases 4–5, $Q = 10^5$ spans test cases 6–10, and the cell $r_i \leq 2.5 \times 10^6$ spans test cases 5–6.)

Problem 2: Zoo (动物园)

Problem file name: `zoo`. Time limit: 1 s. Memory limit: 256 MB.

Description

The zoo keeps many animals. According to the animals being kept, the keeper, Little A, buys different kinds of feed following the Feeding Guide, and sends the shopping list to the purchaser, Little B.

Concretely, there are 2^k different kinds of animals in the animal world, numbered $0 \sim 2^k - 1$. The zoo keeps n of them, and the number of the i-th kept animal is a_i.

The Feeding Guide contains m requirements in total. The j-th requirement has the form "if the zoo keeps some animal whose number has bit p_j equal to 1 in binary representation, then feed number q_j must be bought". There are c kinds of feed in total, numbered $1 \sim c$. In this problem the binary representation of an animal's number is regarded as a k-bit 01-string, where bit 0 is the lowest bit and bit $k - 1$ is the highest bit.

According to the Feeding Guide, Little A draws up a feed list and gives it to Little B, who buys the feed. The list has the form of a c-bit 01-string. When bit i is 1, feed number i needs to be bought; when bit i is 0, feed number i does not need to be bought.

In fact, with the feed that has been bought, the zoo may be able to keep more animals. More specifically, if adding a currently unkept animal with number x to the zoo does not change the feed list, then we consider that the zoo can currently also keep the animal with number x.

Now Little B asks you to help compute how many more kinds of animals the zoo can currently keep.

Input

The input file is named `zoo.in`.

The first line contains four space-separated integers n, m, c, k. They denote, respectively, the number of animals in the zoo, the number of requirements in the Feeding Guide, the number of kinds of feed, and the number of bits in the binary representation of animal numbers.

The second line contains n space-separated integers, the i-th of which is a_i.

Each of the next m lines contains two integers p_i, q_i, describing one requirement.

It is guaranteed that all a_i are pairwise distinct and all q_i are pairwise distinct.

Output

The output file is named `zoo.out`.

A single line containing one integer, the answer.

Sample 1 input

```
3 3 5 4
1 4 6
0 3
2 4
2 5
```

Sample 1 output

```
13
```

Explanation of sample 1

The zoo keeps three kinds of animals, numbered 1, 4, 6. The 3 requirements in the Feeding Guide are:

1. If bit 0 of the number of some kept animal is 1, feed number 3 must be bought.
2. If bit 2 of the number of some kept animal is 1, feed number 4 must be bought.
3. If bit 2 of the number of some kept animal is 1, feed number 5 must be bought.

The feed purchases are:

1. Bit 0 of the number of animal 1 is 1, so feed number 3 must be bought;
2. Bit 2 of the numbers of animals 4 and 6 is 1, so feed numbers 4 and 5 must be bought.

Since adding to the current zoo any one animal with a number among $0, 2, 3, 5, 7, 8, \cdots, 15$ does not change the shopping list, the answer is 13.

Sample 2 input

```
2 2 4 3
1 2
1 3
2 4
```

Sample 2 output

```
2
```

Sample 3

See `zoo/zoo3.in` and `zoo/zoo3.ans` in the contestant's directory. (On QOJ these are given as downloadable files `ex_zoo3.in` and `ex_zoo3.ans`.)

Constraints

For 20% of the data: $k \le n \le 5$, $m \le 10$, $c \le 10$, and all p_i are pairwise distinct.
For 40% of the data: $n \le 15$, $k \le 20$, $m \le 20$, $c \le 20$.
For 60% of the data: $n \le 30$, $k \le 30$, $m \le 1000$.
For 100% of the data: $0 \le n, m \le 10^6$, $0 \le k \le 64$, $1 \le c \le 10^8$.

Problem 3: Function Calls (函数调用)

Problem file name: `call`. Time limit: 2 s. Memory limit: 256 MB.

Description

Functions are an important concept in all kinds of programming languages. With the help of functions, we can always decompose a complex task into relatively simple subtasks, refining them until they become very simple basic operations, so that the code is organized more rigorously and more orderly. However, too many function calls also incur extra overhead and affect the running efficiency of a program.

A certain database application provides a number of functions for maintaining the data. It is known that the functionality of these functions falls into three categories:

1. add a value to a specified element of the data;
2. multiply every element of the data by the same value;
3. perform a number of function calls in sequence; it is guaranteed that no recursion occurs (i.e., no function calls itself directly or indirectly).

When using this database application, the user can input at once the sequence of functions to be called (a function may be called several times); after the functions in the sequence have been executed in order, the data in the system is updated accordingly. One day, Little A ran into trouble while processing data with this database program: due to frequent and inefficient function calls, the system became unresponsive while executing the operations, and he had to force-quit the database program. In order to compute the correct data, Little A consulted the documentation of the software and learned the specific functionality of every function. Now he asks you to help him compute, from this information, what the updated data should be.

Input

The input file is named `call.in`.

Line 1 contains a positive integer n, the number of data items.

Line 2 contains n integers, the i-th of which is the initial value a_i of the data item with index i.

Line 3 contains a positive integer m, the number of functions provided by the database application. The functions are numbered $1 \sim m$.

In the next m lines, the first integer of line j ($1 \leq j \leq m$) is T_j, the type of function number j:

1. If $T_j = 1$, two integers P_j, V_j follow, denoting respectively the index of the element to which the addition is applied and the value added to it;
2. If $T_j = 2$, one integer V_j follows, denoting the value by which all elements are multiplied;
3. If $T_j = 3$, a positive integer C_j follows, denoting the number of functions called by function number j, and then C_j integers $g_1^{(j)}, g_2^{(j)}, \cdots, g_{C_j}^{(j)}$ denote in order the numbers of the functions it calls.

Line $m+4$ contains a positive integer Q, the length of the input sequence of function operations.

Line $m + 5$ contains Q integers f_i, the i-th of which is the number of the i-th function to be executed.

Output

The output file is named `call.out`.

A single line containing n space-separated integers: in the order of the indices $1 \sim n$, output the value of every element in the database after the input sequence of functions has been executed. The answers are taken modulo 998244353.

Sample 1 input

```
3
1 2 3
3
1 1 1
2 2
3 2 1 2
2
2 3
```

Sample 1 output

```
6 8 12
```

Explanation of sample 1

Function number 1 adds one to the value of a_1. Function number 2 multiplies all elements by 2. Function number 3 first calls function number 1, then calls function number 2.

The final function sequence first executes function number 2, and the values of all elements become $2, 4, 6$.

Then function number 3 is executed: it first calls function number 1, and the values of all elements become $3, 4, 6$. It then calls function number 2, and the values of all elements become $6, 8, 12$.

Sample 2 input

```
10
1 2 3 4 5 6 7 8 9 10
8
3 2 2 3
3 2 4 5
3 2 5 8
2 2
3 2 6 7
1 2 5
1 7 6
2 3
3
1 2 3
```

Sample 2 output

```
36 282 108 144 180 216 504 288 324 360
```

Sample 3

See `call/call3.in` and `call/call3.ans` in the contestant's directory. (On QOJ these are given as downloadable files `ex_call3.in` and `ex_call3.ans`.)

Constraints

Test case number	$n, m, Q \leq$	$\sum C_j$	Other special restrictions
$1 \sim 2$	1000	$= m - 1$	the function call relation forms a tree
$3 \sim 4$		≤ 100	none
$5 \sim 6$		≤ 40000	no type-2 functions or no type-1 functions
7	20000	$= 0$	none
$8 \sim 9$		$= m - 1$	the function call relation forms a tree
$10 \sim 11$			none
$12 \sim 13$		$\leq 2 \times 10^5$	no type-2 functions or no type-1 functions
14		$= 0$	none
$15 \sim 16$	10^5	$= m - 1$	the function call relation forms a tree
$17 \sim 18$		$\leq 5 \times 10^5$	none
$19 \sim 20$		$\leq 10^6$	

(In the original table, the cell $n, m, Q \leq 1000$ spans test cases 1–6, $n, m, Q \leq 20000$ spans test cases 7–14, $n, m, Q \leq 10^5$ spans test cases 15–20, the cell $\sum C_j \leq 2 \times 10^5$ spans test cases 10–13, and the last "none" spans test cases 17–20.)

For all data: $0 \leq a_i \leq 10^4$, $T_j \in \{1, 2, 3\}$, $1 \leq P_j \leq n$, $0 \leq V_j \leq 10^4$, $1 \leq g_k^{(j)} \leq m$, $1 \leq f_i \leq m$.

Problem 4: Snakes (贪吃蛇)

Problem file name: `snakes`. Time limit: 2 s. Memory limit: 256 MB.

Description

There are n snakes on the grassland, numbered $1, 2, \cdots, n$. Initially every snake has a strength value a_i. We say that the snake with number x is stronger than the snake with number y if and only if their current strength values satisfy $a_x > a_y$, or $a_x = a_y$ and $x > y$.

Next these snakes will fight a duel, which lasts for a number of rounds. In every round the strongest snake has the right to choose: it may choose to eat, or not to eat, the weakest snake.

1. If it chooses to eat, then the strength value of the strongest snake is decreased by the strength value of the weakest snake, and the weakest snake is eaten and leaves the rest of the duel. Then the next round of the duel begins.

2. If it chooses not to eat, the duel ends immediately.

Every snake wants to eat as many other snakes as possible in the duel, under the premise that it is not eaten itself (obviously, a snake will not choose to eat itself).

Now assume that every snake is sufficiently clever. Please find how many snakes remain after the duel ends.

This problem has multiple test groups. For the first group, the strength of every snake is given completely by the input; for each subsequent group, the strengths of some of the snakes are modified relative to the previous group, and this is given as the new input.

Input

The input file is named `snakes.in`.

The first line contains a positive integer T, the number of test groups.

Then follow T groups of data. For group 1, the first line contains a positive integer n, and the second line contains n non-negative integers a_i.

For groups 2 to T, each group consists of:

the first line, containing a non-negative integer k, the number of snakes whose strength is modified;

the second line, containing $2k$ integers, where every two integers form a pair (x, y), meaning that the value of a_x is changed to y, in order. A position may be modified several times; the last modification is the one that counts.

Output

The output file is named `snakes.out`.

Output T lines, each containing one integer, the number of snakes that finally survive.

Sample 1 input

```
2
3
11 14 14
3
1 5 2 6 3 25
```

Sample 1 output

```
3
1
```

Explanation of sample 1

In the first group of data, in the first round snake 3 is the strongest and snake 1 is the weakest. If snake 3 chooses to eat, it will be eaten by snake 2 in the second round. Therefore snake 3 chooses not to eat in the first round, and all 3 snakes survive.

For the second group of data, the strengths of the 3 snakes become $5, 6, 25$. In the first round snake 3 is the strongest and snake 1 is the weakest. If it chooses to eat, the strength value of snake 3 becomes 20; it is still the strongest snake in the second round and can eat snake 2. Therefore snake 3 chooses to eat in both rounds, and finally only 1 snake survives.

Sample 2 input

```
2
5
13 31 33 39 42
5
1 7 2 10 3 24 4 48 5 50
```

Sample 2 output

```
5
3
```

Sample 3

See `snakes/snakes3.in` and `snakes/snakes3.ans` in the contestant's directory.

Sample 4

See `snakes/snakes4.in` and `snakes/snakes4.ans` in the contestant's directory.

Constraints

For 20% of the data: $n = 3$.

For 40% of the data: $n \leq 10$.

For 55% of the data: $n \leq 2000$.

For 70% of the data: $n \leq 5 \times 10^4$.

For 100% of the data: $3 \leq n \leq 10^6$, $1 \leq T \leq 10$, $0 \leq k \leq 10^5$, $0 \leq a_i, y \leq 10^9$. It is guaranteed that in every group of data (including after all modifications have been made), the a_i are arranged in non-decreasing order.

Source

The statements were taken from the official CSP-S 2020 problem PDFs hosted on QOJ (contest "CSP-S 2020", QOJ contest 538):

- Problem 1: `https://qoj.ac/problem/2013`
- Problem 2: `https://qoj.ac/problem/2014`
- Problem 3: `https://qoj.ac/problem/2015`
- Problem 4: `https://qoj.ac/problem/2016`